

Contents

Rapport MSPR — FutureKawa	3
1. Résumé exécutif	3
2. Introduction	3
3. Cahier des charges et besoins	4
3.1 Besoins fonctionnels	4
3.2 Besoins techniques	5
3.3 Besoins IoT	5
4. Architecture globale	6
4.1 Schéma d'architecture	6
4.2 Flux des données	6
4.3 Choix de l'architecture microservices	7
5. Architecture technique détaillée	7
5.1 Tableau des composants	7
5.2 Frontend	8
5.3 Backend Central	9
5.4 Backends pays	9
5.5 Base de données	9
5.6 MQTT / Mosquitto	9
5.7 Docker	10
5.8 Swagger	10
5.9 Tests	10
5.10 Jenkins	11
5.11 IoT	11
5.12 Authentification JWT	11
5.13 Notification email (MailHog)	12
5.14 Connecteur ERP	12
5.15 Health Check	13
6. Modèle de données	13
6.1 Entités	13
6.2 Schéma des relations	14
6.3 Statuts des lots	14
6.4 Types d'alerte	14
7. Gestion des lots et FIFO	14
7.1 Création de lots	14
7.2 Consultation des lots	15
7.3 Sortie FIFO	15
7.4 Importance métier du FIFO	15
8. Supervision IoT et MQTT	15
8.1 Rôle de MQTT	15
8.2 Rôle de Mosquitto	16
8.3 Chaîne complète de traitement	16
8.4 Format du message MQTT	16
8.5 Commande de test	16
9. Prototype Arduino UNO + DHT11	16
9.1 Montage	16
9.2 Capteur DHT11	17
9.3 Code Arduino	17
9.4 Bridge Serial-to-MQTT	17
9.5 Limites du prototype	17

10. Gestion des alertes	17
10.1 Types d'alertes	17
10.2 Seuils par pays	18
10.3 Génération automatique	18
10.4 Statut des alertes	18
11. Réglage des seuils d'alerte	18
11.1 Fonctionnalité	18
11.2 Endpoints	18
11.3 Exemple de modification	18
11.4 Interface frontend	19
11.5 Limite actuelle	19
12. Frontend dashboard	19
12.1 Présentation	19
12.2 Sections du dashboard	19
12.3 Navigation	20
12.4 Gestion des erreurs	20
13. Documentation API Swagger	20
13.1 Accès Swagger	20
13.2 Intérêt pour les développeurs	20
13.3 Intérêt pour l'intégration externe	20
14. Docker et déploiement local	20
14.1 Démarrage	20
14.2 Arrêt	21
14.3 Vérification	21
14.4 Logs	21
15. CI/CD Jenkins	21
15.1 Pipeline préparé	21
15.2 Rapport de test JUnit	21
15.3 Statut	21
16. Notification email avec MailHog	22
16.1 Présentation	22
16.2 Architecture	22
16.3 Configuration	22
16.4 Endpoints de test	22
16.5 Types d'alertes email	23
16.6 Test rapide	23
17. Intégration ERP / progiciel	23
17.1 Positionnement	23
17.2 Architecture d'intégration	23
17.3 Endpoints utilisables par un ERP	24
17.4 ERP compatibles	24
17.5 Limite	24
18. Sécurité et limites	25
18.1 Limites actuelles	25
18.2 Points forts	25
19. Gestion du changement	25
19.1 Pourquoi un accompagnement est nécessaire	25
19.2 Profils concernés	25
19.3 Plan d'accompagnement	26
19.4 Indicateurs de réussite	26
20. Documentation utilisateur	26

21. Matrice de conformité — Grille d'évaluation	27
22. Preuves à insérer dans le rapport	29
23. Conclusion	30
24. Annexes	31
24.1 Commandes Docker	31
24.2 Commandes MQTT	31
24.3 Endpoints API principaux	31
24.4 Ports utilisés	32
24.5 Seuils par pays	32
24.6 Commandes de test	33
24.7 Commandes Jenkins	33
24.8 Commandes MailHog (si déployé)	33
Checklist finale — Avant soutenance	33

Rapport MSPR — FutureKawa

Projet : FutureKawa — Supervision IoT et gestion des stocks de café vert **Type :** MSPR — Microservices / IoT / Supervision **Année :** 2025-2026 **Groupe / équipe :** À compléter **Auteur(s) :** À compléter **Version du rapport :** 1.0

1. Résumé exécutif

FutureKawa est une plateforme de supervision des stocks de café vert dans des entrepôts situés dans trois pays : Brésil, Équateur et Colombie. Le problème métier est simple : le café vert est une marchandise sensible à la température et à l'humidité. Sans supervision en temps réel, des lots peuvent être dégradés ou perdus sans que l'équipe qualité ne le détecte à temps. La solution FutureKawa repose sur une architecture microservices avec un backend central qui agrège les données de trois services pays indépendants. Chaque pays possède sa propre base PostgreSQL, son propre broker MQTT Mosquitto, et son propre backend NestJS. Un frontend React/Vite permet de visualiser les lots, les mesures IoT, les alertes et de configurer les seuils. Un prototype IoT réel basé sur Arduino UNO + capteur DHT11 collecte la température et l'humidité, transmises via un bridge Serial-to-MQTT Node.js jusqu'au dashboard. Les résultats obtenus incluent 104 tests automatisés validés, une documentation complète (technique, utilisateur, API Swagger, ERP), un pipeline CI/CD Jenkins préparé, une authentification JWT avec rôles ADMIN/OPERATOR, un système de notification email via MailHog, un connecteur ERP simulé, un endpoint de health check, et un déploiement Docker Compose fonctionnel avec 12 conteneurs. Les limites actuelles concernent le stockage en mémoire des seuils d'alerte et l'intégration ERP qui reste partielle. Les évolutions futures incluent la migration vers ESP32, le déploiement cloud, et l'ajout de connecteurs ERP réels.

2. Introduction

Le café vert est une matière première agricole qui doit être stockée dans des conditions précises de température et d'humidité pour préserver sa qualité. Une

conservation inadéquate peut entraîner des pertes financières importantes pour les entreprises de distribution de café.

FutureKawa répond à plusieurs besoins métier :

- **Traçabilité des lots** : chaque lot de café est identifié par un code unique, avec sa date d'entrée, son entrepôt de stockage et son statut (conforme, en alerte, périmé).
- **Supervision température/humidité** : des capteurs IoT (ou simulations) mesurent en continu les conditions de stockage et transmettent les données via MQTT.
- **Alertes qualité** : si un seuil de température ou d'humidité est dépassé, une alerte est créée automatiquement. Les lots stockés depuis plus d'un an sont également signalés comme périmés.
- **Solution multi-pays** : FutureKawa couvre trois pays producteurs avec des seuils adaptés aux conditions climatiques locales de chaque région.

L'objectif du projet est de fournir un outil de supervision complet, déployable localement avec Docker, documenté, testé, et prêt à être intégré dans un système d'information plus large.

3. Cahier des charges et besoins

3.1 Besoins fonctionnels

Besoin	Réponse FutureKawa	Statut
Gérer les lots de café par pays	API CRUD complète avec création, liste, statut	Réalisé
Consulter les lots par pays	Endpoint GET <code>/api/central/:country/lots</code>	Réalisé
Sortie FIFO des lots	Endpoint GET <code>/api/central/:country/lots/fifo</code>	Réalisé
Collecter les mesures température/humidité	Abonnement MQTT + stockage PostgreSQL	Réalisé
Publier des mesures MQTT	Scripts de simulation + bridge Arduino	Réalisé
Créer des alertes automatiques	Alertes température, humidité, lots périmés	Réalisé
Afficher un dashboard frontend	React/Vite avec graphiques et tableaux	Réalisé
Sélectionner un pays	Sélecteur dans la barre de navigation	Réalisé
Régler les seuils d'alerte depuis le frontend	Interface dédiée avec reset	Réalisé

Besoin	Réponse FutureKawa	Statut
Documenter les API	Swagger sur chaque backend	Réalisé
Authentification et rôles	JWT avec rôles ADMIN/OPERATOR	Réalisé
Notification email	MailHog SMTP avec EmailService	Réalisé
Connecteur ERP	Module ERP simulé (lots, alertes, sync)	Réalisé
Health check	Endpoint GET /api/health	Réalisé
Export de données	Non implémenté	Prévu

3.2 Besoins techniques

Besoin	Réponse FutureKawa	Statut
Architecture microservices	Backend central + 3 services pays	Réalisé
Base de données par pays	PostgreSQL avec Prisma ORM	Réalisé
Broker IoT par pays	Mosquitto MQTT	Réalisé
Conteneurisation	Docker Compose (12 conteneurs)	Réalisé
Tests automatisés	104 tests (Jest + Vitest)	Réalisé
CI/CD	Jenkinsfile préparé	Partiel
Documentation utilisateur	Guide complet	Réalisé
Documentation technique	Architecture, choix techniques, déploiement	Réalisé
Documentation API	Swagger sur tous les backends	Réalisé
Intégration ERP	Module ERP simulé (endpoints dédiés)	Réalisé
Notification email	MailHog + EmailService (SMTP)	Réalisé

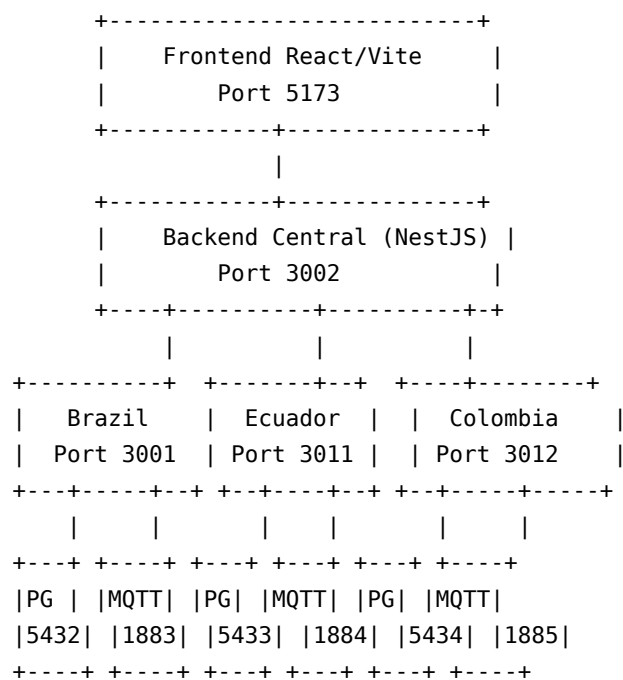
3.3 Besoins IoT

Besoin	Réponse FutureKawa	Statut
Capteur température réel	Arduino UNO + DHT11	Réalisé
Transmission des mesures	Bridge Serial-to-MQTT Node.js	Réalisé

Besoin	Réponse FutureKawa	Statut
Simulation sans matériel	Scripts MQTT de simulation	Réalisé
Stockage des mesures	PostgreSQL via backend pays	Réalisé
Visualisation temps réel	Dashboard frontend avec graphiques	Réalisé
Alertes automatiques	Création d'alerte si seuil dépassé	Réalisé
Wi-Fi intégré (ESP32)	Non implémenté	Prévu

4. Architecture globale

4.1 Schéma d'architecture



4.2 Flux des données

Flux IHM (consultation) :

Frontend → Backend Central → Service Pays → PostgreSQL → Réponse

Flux IoT (capteurs) :

Arduino DHT11 / Simulation MQTT → Mosquitto → Service Pays → PostgreSQL

↓
Backend Central
↓

4.3 Choix de l'architecture microservices

L'architecture microservices a été choisie pour plusieurs raisons :

- **Séparation par pays** : chaque pays a son propre backend, sa base de données et son broker MQTT. Cela permet une isolation totale des données et un déploiement indépendant.
- **Isolation des données** : les données du Brésil ne sont pas mélangées avec celles de l'Équateur ou de la Colombie. Chaque base PostgreSQL est indépendante.
- **Maintenabilité** : chaque service peut être mis à jour ou corrigé sans impacter les autres. Le code est organisé par modules (lots, mesures, alertes, MQTT).
- **Évolutivité** : il est possible d'ajouter un nouveau pays (ex. Pérou) en créant un nouveau service sans modifier les services existants.
- **API centralisée** : le frontend n'interroge qu'un seul point d'entrée (backend central), qui route les appels vers les services pays. Cela simplifie le développement frontend.

5. Architecture technique détaillée

5.1 Tableau des composants

Composant	Technologie	Rôle
Frontend	React 19 + TypeScript + Vite	Interface utilisateur du dashboard
Backend Central	NestJS (Node.js)	Passerelle d'agrégation des données
Brazil Service	NestJS (Node.js)	Gestion des lots, mesures, alertes Brésil
Ecuador Service	NestJS (Node.js)	Gestion des lots, mesures, alertes Équateur
Colombia Service	NestJS (Node.js)	Gestion des lots, mesures, alertes Colombie
PostgreSQL	15-alpine	Base de données relationnelle par pays

Composant	Technologie	Rôle
Mosquitto MQTT	eclipse-mosquitto:2	Broker IoT par pays
Prisma	ORM JavaScript	Accès aux bases de données
Swagger	@nestjs/swagger	Documentation interactive des API
Docker Compose	Docker	Déploiement local conteneurisé
Jest	Framework de test	Tests backend
Vitest + RTL	Framework de test	Tests frontend
Jenkins	CI/CD	Pipeline d'intégration continue
JWT + Passport	Authentification	Auth JWT avec rôles ADMIN/OPERATOR
Nodemailer	Email	Envoi d'alertes via MailHog SMTP
Arduino UNO + DHT11	Matériel	Capteur IoT température/humidité
Bridge Serial-to-MQTT	Node.js + serialport	Transmission série vers MQTT

5.2 Frontend

Le frontend est une application React 19 avec TypeScript, construite avec Vite. Il se compose de :

- **Dashboard** : page principale avec vue d'ensemble, graphiques et tableaux
- **Navbar** : barre de navigation avec sélecteur de pays
- **CountryOverview** : indicateurs clés (nombre de lots, mesures, alertes)
- **DashboardCards** : cartes récapitulatives
- **MeasurementsChart** : graphique d'évolution température/humidité
- **AlertsChart** : graphique circulaire de répartition des alertes
- **LotsTable** : tableau des lots en stock
- **FifoLotsTable** : tableau des lots triés par FIFO
- **MeasurementsTable** : tableau des mesures capteurs
- **AlertsTable** : tableau des alertes
- **ThresholdSettings** : interface de réglage des seuils d'alerte

Le frontend communique avec le backend central via `VITE_API_BASE_URL` (port 3002).

5.3 Backend Central

Le backend central est un service NestJS qui agit comme passerelle d'agrégation. Il expose :

- GET /api/central/countries — liste des pays disponibles
- GET /api/central/:country/lots — lots d'un pays
- GET /api/central/:country/lots/fifo — lots triés par FIFO
- GET /api/central/:country/measurements — mesures d'un pays
- GET /api/central/:country/alerts — alertes d'un pays
- GET /api/central/:country/thresholds — seuils d'alerte
- PUT /api/central/:country/thresholds — modification des seuils
- POST /api/central/:country/thresholds/reset — réinitialisation des seuils

Il interroge chaque service pays via HTTP et fédère les réponses.

5.4 Backends pays

Chaque backend pays (Brazil, Ecuador, Colombia) est un service NestJS complet avec :

- **LotsModule** : gestion des lots de café (création, liste, FIFO, statut)
- **MeasurementsModule** : gestion des mesures capteurs (création, historique)
- **AlertsModule** : gestion des alertes (création automatique, liste, lots périmés)
- **MqttModule** : abonnement MQTT et traitement des messages en temps réel
- **ThresholdsModule** : gestion des seuils d'alerte configurables
- **PrismaModule** : accès à la base de données PostgreSQL

Chaque service pays expose Swagger sur /api/docs.

5.5 Base de données

Chaque pays dispose de sa propre instance PostgreSQL avec trois tables :

- **CoffeeLot** : lots de café (code, pays, entrepôt, date, statut)
- **SensorMeasurement** : mesures capteurs (entrepôt, température, humidité, date)
- **Alert** : alertes (type, message, statut, date)

L'accès aux données se fait via Prisma ORM, qui fournit un typage fort et une génération automatique du client TypeScript.

5.6 MQTT / Mosquitto

Chaque pays possède un broker Mosquitto dédié. Les topics MQTT sont :

- futurekawa/brazil/warehouse-1/measurements (port 1883)
- futurekawa/ecuador/warehouse-1/measurements (port 1884)
- futurekawa/colombia/warehouse-1/measurements (port 1885)

Chaque backend pays s'abonne au topic de son pays et traite les messages en temps réel.

5.7 Docker

L'application complète est déployée avec Docker Compose. Elle comprend 12 conteneurs :

Container	Port	Rôle
postgres-brazil	5432	Base de données Brésil
postgres-ecuador	5433	Base de données Équateur
postgres-colombia	5434	Base de données Colombie
mosquitto-brazil	1883	Broker MQTT Brésil
mosquitto-ecuador	1884	Broker MQTT Équateur
mosquitto-colombia	1885	Broker MQTT Colombie
brazil-service	3001	Backend Brésil
ecuador-service	3011	Backend Équateur
colombia-service	3012	Backend Colombie
backend-central	3002	Backend central
mailhog	1025 SMTP / 8025 UI	Email test SMTP
frontend	5173	Interface utilisateur

Les conteneurs sont interconnectés via un réseau Docker interne. Les données sont persistées avec des volumes Docker. Le 12e conteneur (mailhog) permet la capture et la visualisation des emails d'alerte via une interface web.

5.8 Swagger

Chaque backend expose une documentation Swagger interactive :

- Backend central : <http://localhost:3002/api/docs>
- Brazil : <http://localhost:3001/api/docs>
- Ecuador : <http://localhost:3011/api/docs>
- Colombia : <http://localhost:3012/api/docs>

Swagger permet de tester chaque endpoint directement depuis le navigateur (bouton "Try it out").

5.9 Tests

Les tests sont répartis comme suit :

Service	Nombre de tests	Statut
Brazil Service	19	Réalisé
Ecuador Service	17	Réalisé
Colombia Service	14	Réalisé
Backend Central	23	Réalisé
Frontend	31	Réalisé

Service	Nombre de tests	Statut
Total	104	Réalisé

Les tests backend utilisent Jest (@nestjs/testing). Les tests frontend utilisent Vitest et React Testing Library.

5.10 Jenkins

Le pipeline CI/CD Jenkins est préparé avec un Jenkinsfile complet. Il exécute les étapes suivantes :

1. **Checkout** : récupération du code source
2. **Environment Information** : affichage des versions Node.js, npm, Docker
3. **Install Dependencies** : installation parallèle des dépendances (5 services)
4. **Generate Prisma Clients** : génération des clients Prisma pour les 3 pays
5. **Run Backend Tests** : exécution parallèle des tests backend (4 services)
6. **Run Frontend Tests** : exécution des tests frontend
7. **Build Frontend** : build de production du frontend
8. **Validate Docker Compose** : validation de la configuration Docker Compose
9. **Docker Build Check** (optionnel) : build des images Docker

Statut : le Jenkinsfile est prêt et documenté. La preuve de fonctionnement (build réussi, capture d'écran) reste à compléter.

5.11 IoT

Le prototype IoT se compose de :

- **Arduino UNO** : carte microcontrôleur
- **Capteur DHT11** : mesure de température et humidité
- **Connexion USB série** : transmission des données à 9600 bauds
- **Bridge Serial-to-MQTT** : script Node.js qui lit le port série et publie vers Mosquitto
- **Backend Colombia** : abonné au topic MQTT Colombia, stocke les mesures en base
- **Affichage dashboard** : visualisation des mesures et alertes dans le frontend

Chaîne IoT complète :

```

Arduino UNO + DHT11
→ USB Serial (9600 bauds)
→ Node.js Serial-to-MQTT bridge
→ Mosquitto Colombia (port 1885)
→ Colombia backend (NestJS)
→ PostgreSQL Colombia
→ Backend Central (port 3002)
→ Frontend dashboard (port 5173)

```

5.12 Authentification JWT

Le backend central dispose d'un module d'authentification JWT complet :

- **AuthModule** : configuration JWT avec secret et expiration 24h
- **AuthController** : POST /api/auth/login avec validation des credentials
- **JwtStrategy** : validation des tokens via Passport
- **RolesGuard** : protection des endpoints sensibles par rôle
- **JwtAuthGuard** : protection simple (authentification seule)

Rôles disponibles :

Rôle	Accès
ADMIN	Accès complet à tous les endpoints
OPERATOR	Accès limité (consultation uniquement)

Utilisateurs mockés pour la démonstration :

Utilisateur	Mot de passe	Rôle
admin	admin123	ADMIN
operator	operator123	OPERATOR

Endpoints protégés nécessitant le rôle ADMIN : - PUT /api/central/:country/thresholds
- POST /api/central/:country/thresholds/reset

5.13 Notification email (MailHog)

Le backend central dispose d'un service d'envoi d'emails via MailHog :

- **EmailModule** : configuration Nodemailer avec SMTP MailHog
- **EmailService** : méthodes d'envoi pour chaque type d'alerte
- **EmailController** : endpoints de test pour la démonstration

Les emails sont envoyés lors des événements suivants : - Alerte température dépassée (TEMPERATURE) - Alerte humidité dépassée (HUMIDITY) - Alerte lot périmé (EXPIRED_LOT)

5.14 Connecteur ERP

Le backend central expose un module ERP qui simule une intégration avec des systèmes externes :

- **ErpModule** : connecteur avec endpoints dédiés
- **ErpController** : endpoints aux formats standardisés
- **ErpService** : agrégation des données multi-pays

Le connecteur ERP permet à un système externe de récupérer les données FutureKawa dans un format standardisé, comme si l'application était intégrée à un ERP.

5.15 Health Check

Le backend central expose un endpoint de monitoring pour vérifier l'état de santé du système :

- **HealthModule** : service de vérification
- **HealthController** : GET /api/health
- **HealthService** : vérification de tous les services pays

Le rapport de santé inclut : - Statut global (healthy / degraded) - Statut de chaque service pays (available / unavailable) - Latence de chaque service - Uptime du backend central - Utilisation mémoire

6. Modèle de données

6.1 Entités

CoffeeLot (lot de café)

Champ	Type	Description
id	Int (auto-increment)	Identifiant unique
lotCode	String (unique)	Code du lot
country	String	Pays (ex. "Brazil")
warehouseName	String	Nom de l'entrepôt
storageDate	DateTime	Date d'entrée en stock
status	LotStatus	CONFORME, EN_ALERTE, PERIME
createdAt	DateTime	Date de création

SensorMeasurement (mesure capteur)

Champ	Type	Description
id	Int (auto-increment)	Identifiant unique
warehouseName	String	Nom de l'entrepôt
temperature	Float	Température en °C
humidity	Float	Humidité en %
measuredAt	DateTime	Date et heure de la mesure

Alert (alerte)

Champ	Type	Description
id	Int (auto-increment)	Identifiant unique
type	AlertType	TEMPERATURE, HUMIDITY, EXPIRED_LOT
message	String	Description de l'alerte

Champ	Type	Description
status	String	"NEW" par défaut
createdAt	DateTime	Date de création

6.2 Schéma des relations

CoffeeLot	SensorMeasurement	Alert
id (PK)	id (PK)	id (PK)
lotCode (unique)	warehouseName	type (enum)
country	temperature	message
warehouseName	humidity	status
storageDate	measuredAt	createdAt
status (enum)		
createdAt		

Les trois tables sont indépendantes (pas de clés étrangères). La liaison se fait par le nom d'entrepôt (warehouseName) dans la logique métier.

6.3 Statuts des lots

Statut	Description
CONFORME	Le lot est dans les conditions normales
EN_ALERTE	Une alerte température ou humidité est active pour cet entrepôt
PERIME	Le lot est stocké depuis plus de 365 jours

6.4 Types d'alerte

Type	Déclencheur
TEMPERATURE	Température hors seuil
HUMIDITY	Humidité hors seuil
EXPIRED_LOT	Lot stocké depuis plus d'un an

7. Gestion des lots et FIFO

7.1 Création de lots

Les lots de café sont créés via une requête POST sur le backend pays :

```
POST /api/lots
Content-Type: application/json

{
  "lotCode": "BRA-2025-001",
  "country": "Brazil",
  "warehouseName": "Warehouse Brazil 1",
  "storageDate": "2025-01-15T00:00:00.000Z"
}
```

Chaque lot reçoit un code unique et un statut initial "CONFORME".

7.2 Consultation des lots

Les lots peuvent être consultés par pays via le backend central :

```
GET /api/central/:country/lots
```

Exemple : GET /api/central/brazil/lots

Retourne la liste des lots du pays triés du plus récent au plus ancien.

7.3 Sortie FIFO

La sortie FIFO (First In, First Out) est une règle de gestion des stocks qui consiste à sortir en priorité les lots les plus anciens.

```
GET /api/central/:country/lots/fifo
```

Exemple : GET /api/central/brazil/lots/fifo

Retourne les lots triés par date d'entrée croissante (du plus ancien au plus récent).

7.4 Importance métier du FIFO

Le respect de la règle FIFO est essentiel dans le stockage du café vert car :

- Il garantit que les lots les plus anciens sont expédiés en premier
- Il réduit le risque de lots périmés (stockage > 1 an)
- Il préserve la qualité du café en limitant le temps de stockage
- Il est conforme aux bonnes pratiques de gestion d'entrepôt

8. Supervision IoT et MQTT

8.1 Rôle de MQTT

MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie léger, standard pour l'IoT. Il fonctionne selon un modèle publish/subscribe où : - Les **capteurs publient** des messages sur des topics - Les **backends s'abonnent** aux topics pour recevoir les messages en temps réel

8.2 Rôle de Mosquitto

Mosquitto est un broker MQTT open-source. FutureKawa utilise un broker Mosquitto par pays pour isoler les flux IoT de chaque pays.

8.3 Chaîne complète de traitement

Publication MQTT → Réception broker → Traitement backend → Stockage PostgreSQL → Création alerte si nécessaire

1. Un capteur (ou script de simulation) publie une mesure sur le topic MQTT du pays
2. Le broker Mosquitto reçoit le message
3. Le backend pays, abonné au topic, traite le message
4. La mesure est stockée dans PostgreSQL
5. Si la température ou l'humidité dépasse les seuils configurés, une alerte est créée
6. Le frontend affiche les mesures et alertes en temps réel

8.4 Format du message MQTT

```
{
  "warehouseName": "Warehouse Brazil 1",
  "temperature": 27.5,
  "humidity": 55.0
}
```

8.5 Commande de test

Pour simuler une mesure depuis le terminal (ex. pour la Colombie avec des valeurs hors seuil) :

```
mosquitto_pub -h localhost -p 1885 \
  -t "futurekawa/colombia/warehouse-1/measurements" \
  -m '{"warehouseName":"Warehouse Colombia 1","temperature":40,"humidity":20}'
```

Une température de 40°C dépasse le seuil Colombie (30°C - 36°C). Une alerte TEMPERATURE sera automatiquement créée.

9. Prototype Arduino UNO + DHT11

9.1 Montage

Le prototype utilise un Arduino UNO connecté à un capteur DHT11 :

Broche DHT11	Connexion Arduino
+ (alimentation)	5V
OUT (données)	D7 (pin 7)
- (masse)	GND

9.2 Capteur DHT11

Le DHT11 est un capteur numérique bas de gamme qui mesure : - La température de 0°C à 50°C (précision $\pm 2^\circ\text{C}$) - L'humidité de 20% à 80% (précision $\pm 5\%$)

9.3 Code Arduino

Le code embarqué (`futurekawa_dht11_serial.ino`) lit la température et l'humidité toutes les 500 ms et les affiche sur le port série à 9600 bauds :

Humidity: 42.00% Temperature: 27.50°C

9.4 Bridge Serial-to-MQTT

L'Arduino UNO ne dispose pas de module Wi-Fi intégré. Il ne peut donc pas publier directement des messages MQTT. Pour résoudre ce problème, un bridge Serial-to-MQTT en Node.js lit les données du port série USB et les publie vers le broker Mosquitto Colombia.

Architecture du bridge :

Arduino UNO + DHT11 → USB Serial → Node.js bridge → Mosquitto MQTT

Le script (`serial-to-mqtt.js`) utilise les bibliothèques : - `serialport` : lecture du port série - `mqtt` : publication vers Mosquitto - `dotenv` : configuration

9.5 Limites du prototype

- **Précision limitée** : le DHT11 n'est pas un capteur professionnel. Sa précision est de $\pm 2^\circ\text{C}$ pour la température et $\pm 5\%$ pour l'humidité.
- **Bridge nécessaire** : l'Arduino UNO ne peut pas publier MQTT directement sans passerelle USB.
- **Démonstratif** : le prototype a une valeur démonstrative. En production, des capteurs professionnels avec ESP32 (Wi-Fi intégré) seraient recommandés.
- **Port COM** : le bridge nécessite que le port série soit libre (le Serial Monitor Arduino IDE doit être fermé).

10. Gestion des alertes

10.1 Types d'alertes

Type	Condition	Message exemple
TEMPERATURE	Température hors seuil	"Temperature alert: 40°C at Warehouse Colombia 1"
HUMIDITY	Humidité hors seuil	"Humidity alert: 20% at Warehouse Colombia 1"
EXPIRED_LOT	Lot stocké > 365 jours	"Lot BRA-2024-001 is expired (storage > 365 days)"

10.2 Seuils par pays

Pays	Température min	Température max	Humidité min	Humidité max
Brazil	26°C	32°C	53%	57%
Ecuador	28°C	34°C	58%	62%
Colombia	30°C	36°C	30%	45%

Note : les seuils Colombia (30°C-36°C, 30%-45%) sont adaptés à la démonstration locale avec le capteur Arduino DHT11.

10.3 Génération automatique

Les alertes sont générées automatiquement dans deux cas :

1. **Lors de la réception d'une mesure MQTT** : si la température ou l'humidité est hors seuil, une alerte TEMPERATURE ou HUMIDITY est créée.
2. **Lors de la vérification périodique des lots** : les lots stockés depuis plus de 365 jours reçoivent le statut PERIME et une alerte EXPIRED_LOT est créée.

10.4 Statut des alertes

Chaque alerte possède un statut : - NEW : alerte non encore traitée - Les statuts supplémentaires (ACKNOWLEDGED, RESOLVED) sont prévus pour une évolution future

11. Réglage des seuils d'alerte

11.1 Fonctionnalité

Le frontend permet de modifier les seuils d'alerte pour chaque pays directement depuis l'interface utilisateur. Cette fonctionnalité est accessible via le composant **ThresholdSettings**.

11.2 Endpoints

Méthode	URL	Description
GET	/api/central/:country/thresholds	Consulter les seuils actuels
PUT	/api/central/:country/thresholds	Modifier les seuils
POST	/api/central/:country/thresholds	Réinitialiser aux valeurs par défaut

11.3 Exemple de modification

PUT /api/central/colombia/thresholds
Content-Type: application/json

```

{
  "temperature": {
    "min": 31,
    "max": 35
  },
  "humidity": {
    "min": 32,
    "max": 44
  }
}

```

11.4 Interface frontend

Le composant ThresholdSettings affiche : - Les seuils actuels de température (min/max) - Les seuils actuels d'humidité (min/max) - Des champs de saisie pour modifier chaque seuil - Un bouton "Enregistrer" pour sauvegarder - Un bouton "Reset" pour revenir aux valeurs par défaut

11.5 Limite actuelle

Les seuils sont stockés en mémoire (dans le service NestJS). Au redémarrage d'un conteneur, les seuils reviennent aux valeurs par défaut.

Évolution prévue : stockage des seuils en base de données avec gestion des rôles (admin seulement).

12. Frontend dashboard

12.1 Présentation

Le dashboard FutureKawa est l'interface principale de l'application. Il est accessible à <http://localhost:5173>.

12.2 Sections du dashboard

Section	Description
Barre de navigation	Sélecteur de pays (Brazil, Ecuador, Colombia)
Vue d'ensemble	Indicateurs clés : lots, mesures, alertes
Température & Humidité	Graphique d'évolution des mesures dans le temps
Répartition des alertes	Graphique circulaire par type d'alerte
Lots en stock	Tableau détaillé des lots
Sortie FIFO	Liste des lots triés par date (FIFO)
Mesures capteurs	Tableau chronologique des mesures
Alertes	Tableau des alertes avec type et message
Seuils d'alerte	Interface de réglage des seuils

12.3 Navigation

Le sélecteur de pays dans la barre de navigation permet de basculer entre les trois pays. Le dashboard se met à jour automatiquement avec les données du pays sélectionné.

12.4 Gestion des erreurs

Si un backend est indisponible, le dashboard affiche un message d'erreur clair : "Unable to load data from central backend". Les données déjà chargées restent visibles.

13. Documentation API Swagger

13.1 Accès Swagger

Chaque backend NestJS expose une documentation Swagger accessible via navigateur :

Service	URL Swagger
Backend Central	http://localhost:3002/api/docs
Brazil Service	http://localhost:3001/api/docs
Ecuador Service	http://localhost:3011/api/docs
Colombia Service	http://localhost:3012/api/docs

13.2 Intérêt pour les développeurs

Swagger permet aux développeurs de : - Voir la liste complète des endpoints disponibles - Lire la description, les paramètres et les types de réponse de chaque endpoint - Tester les appels API directement depuis le navigateur avec "Try it out" - Voir les codes de retour HTTP et les messages d'erreur

13.3 Intérêt pour l'intégration externe

Pour un ERP ou un système externe, Swagger fournit une documentation complète et standardisée (OpenAPI) qui facilite le développement d'un connecteur. La spécification JSON est accessible via `/api/docs.json`.

14. Docker et déploiement local

14.1 Démarrage

```
docker compose up -d --build
```

Cette commande construit et démarre les 12 conteneurs.

14.2 Arrêt

`docker compose down`

Pour supprimer les volumes (réinitialisation complète) :

`docker compose down -v`

14.3 Vérification

`docker ps`

14.4 Logs

Logs d'un service spécifique

`docker compose logs brazil-service`

`docker compose logs backend-central`

`docker compose logs frontend`

Logs en temps réel

`docker compose logs -f`

15. CI/CD Jenkins

15.1 Pipeline préparé

Le fichier `Jenkinsfile` à la racine du projet définit un pipeline CI/CD complet avec les étapes suivantes :

1. **Checkout** : récupération du code depuis le dépôt Git
2. **Environment Information** : affichage des versions (Node.js, npm, Docker)
3. **Install Dependencies** : installation parallèle des dépendances pour les 5 services
4. **Generate Prisma Clients** : génération des clients Prisma pour les 3 backends pays
5. **Run Backend Tests** : exécution parallèle des tests (4 services backend)
6. **Run Frontend Tests** : exécution des tests frontend
7. **Build Frontend** : build de production du frontend
8. **Validate Docker Compose** : validation de la syntaxe `docker-compose.yml`
9. **Docker Build Check** (optionnel) : build des images Docker (activé avec `BUILD_DOCKER=true`)

15.2 Rapport de test JUnit

Chaque étape de test publie ses résultats au format JUnit, ce qui permet à Jenkins de générer des rapports de test historiques.

15.3 Statut

Le `Jenkinsfile` est préparé, documenté, et prêt à être utilisé sur un serveur Jenkins. La preuve de fonctionnement (build réussi, capture d'écran du pipeline et de la console) reste à compléter.

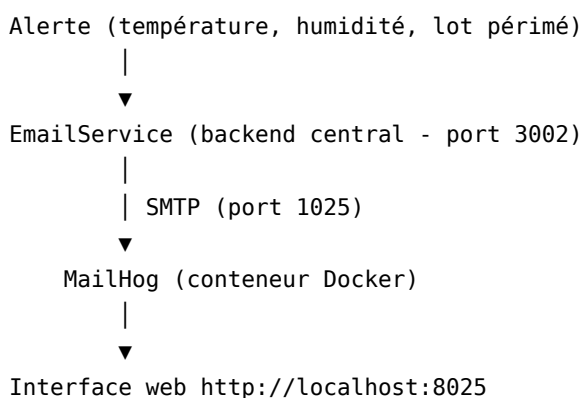
16. Notification email avec MailHog

16.1 Présentation

MailHog est un serveur SMTP de test qui capture les emails envoyés par l'application et les rend visibles dans une interface web.

FutureKawa utilise MailHog pour démontrer l'envoi d'alertes par email. Le backend central contient un **EmailService** (NestJS + Nodemailer) qui envoie des emails via le serveur SMTP MailHog.

16.2 Architecture



16.3 Configuration

MailHog est défini dans le `docker-compose.yml` :

```
mailhog:
  image: mailhog/mailhog:latest
  container_name: mailhog
  ports:
    - "1025:1025" # SMTP
    - "8025:8025" # Interface web
```

Le backend central utilise les variables d'environnement : - MAILHOG_HOST=mailhog - MAILHOG_PORT=1025

16.4 Endpoints de test

Méthode	URL	Description
GET	/api/central/email/test	Envoyer un email de test via MailHog
GET	/api/central/email/simulate/temp	Simuler une alerte température
GET	/api/central/email/simulate/humidite	Simuler une alerte humidité

16.5 Types d'alertes email

Type d'alerte	Destinataire	Objet
Température	pays@futurekawa.com	ALERTE Température - Pays - Entrepôt
Humidité	pays@futurekawa.com	ALERTE Humidité - Pays - Entrepôt
Lot périmé	pays@futurekawa.com	ALERTE Lot Périmé - Pays - Code

16.6 Test rapide

1. Vérifier que MailHog tourne

```
docker ps | grep mailhog
```

2. Envoyer un email test

```
curl http://localhost:3002/api/central/email/test-alert
```

3. Voir le résultat

Ouvrir <http://localhost:8025>

Preuve à compléter : capture de l'interface MailHog avec un email d'alerte reçu dans la boîte de réception.

17. Intégration ERP / progiciel

17.1 Positionnement

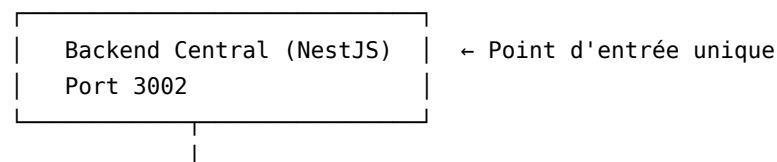
FutureKawa **n'est pas un ERP complet**. Il ne couvre pas la comptabilité, les achats, les ventes ou la paie.

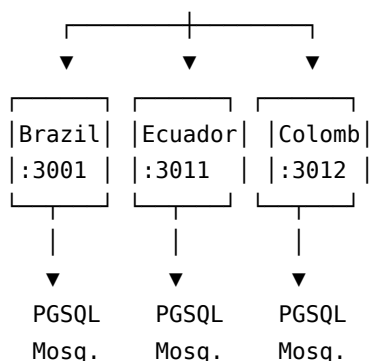
FutureKawa est une **application métier spécialisée** dans la supervision des stocks de café vert. Il peut être utilisé comme **brique intégrable** dans un système d'information plus large, grâce à ses API REST documentées.

17.2 Architecture d'intégration

ERP / Progiciel externe

|
| API REST (HTTP/JSON)





L'ERP externe interroge uniquement le **backend central** (port 3002). Ce dernier route les appels vers les services pays concernés.

17.3 Endpoints utilisables par un ERP

Besoin ERP	Endpoint FutureKawa	Utilité
Consulter les pays disponibles	GET /api/central/countries	Liste des pays avec statut
Consulter les lots par pays	GET /api/central/:country/lots	Suivi des stocks de café
Consulter la sortie FIFO	GET /api/central/:country/lots/fifo	Priorisation des expéditions
Consulter les mesures IoT	GET /api/central/:country/measurements	Conditions de stockage
Consulter les alertes qualité	GET /api/central/:country/alerts	Non-conformités
Consulter les seuils	GET /api/central/:country/thresholds	Configuration qualité
Modifier les seuils	PUT /api/central/:country/thresholds	Ajustement des tolérances

17.4 ERP compatibles

- **Odoo** : ERP open-source modulaire, pourrait consommer les API FutureKawa via un module personnalisé
- **SAP** : pourrait intégrer FutureKawa via des appels REST ou une interface RFC
- **Microsoft Dynamics** : pourrait consommer les données FutureKawa via ses connecteurs HTTP

17.5 Limite

L'intégration ERP réelle n'est pas développée. L'architecture API-first est conçue pour faciliter cette évolution, mais aucun connecteur prêt à l'emploi n'est encore créé. Les endpoints ne sont pas protégés par authentification.

18. Sécurité et limites

18.1 Limites actuelles

Point	Description	Évolution prévue
Authentification	JWT avec rôles ADMIN/OPERATOR	Enrichissement des rôles et permissions
Secrets manager	Les mots de passe sont dans docker-compose.yml	Utilisation de secrets Docker ou vault
Seuils en mémoire	Les seuils d'alerte sont perdus au redémarrage	Stockage en base de données
MailHog	Notification email implémentée via MailHog SMTP	Templates HTML et file d'attente
Jenkins	Pipeline préparé mais preuve à compléter	Capture d'écran du build réussi
Déploiement cloud	Pas encore de déploiement sur AWS/Azure/GCP	Déploiement avec RDS et ECS
Intégration ERP	Module ERP simulé (endpoints API dédiés)	Connecteur Odoo ou SAP réel
Bridge IoT	Bridge USB nécessaire pour Arduino UNO	Migration vers ESP32

18.2 Points forts

- Architecture microservices bien séparée par pays
- Données isolées (base PostgreSQL par pays)
- Documentation complète (technique, utilisateur, API)
- Tests automatisés (104 tests)
- Authentification JWT avec rôles ADMIN/OPERATOR
- Notification email via MailHog SMTP
- Connecteur ERP simulé avec endpoints dédiés
- Endpoint health check et monitoring
- Déploiement reproductible avec Docker Compose (12 conteneurs)
- Interface de réglage des seuils dans le frontend
- Prototype IoT fonctionnel et documenté

19. Gestion du changement

19.1 Pourquoi un accompagnement est nécessaire

Le passage d'un suivi manuel (relevés papier, Excel) à une supervision numérique automatisée représente un changement important. Sans accompagnement, les risques sont : outil sous-utilisé, alertes ignorées, retour aux anciennes méthodes.

19.2 Profils concernés

Profil	Rôle	Niveau technique
Responsables d'entrepôt	Utilisateurs principaux du dashboard	Faible à moyen
Équipes qualité	Surveillance des seuils et alertes	Moyen
Équipes logistiques	Gestion FIFO et expédition	Faible à moyen
Administrateurs techniques	Maintenance du système	Élevé
Direction	Supervision globale	Faible

19.3 Plan d'accompagnement

Informer → Communiquer → Former → Impliquer → Accompagner

1. **Informer** : présentation du projet, objectifs, calendrier
2. **Communiquer** : mise à jour régulière, démonstrations
3. **Former** : sessions adaptées à chaque profil, guide utilisateur
4. **Impliquer** : ambassadeurs dans chaque équipe, retours terrain
5. **Accompagner** : support technique, suivi à J+30, J+60, J+90

19.4 Indicateurs de réussite

Indicateur	Cible
Taux d'utilisation du dashboard	> 80% des utilisateurs
Alertes traitées dans l'heure	> 90%
Réduction des lots périmés	> 50% en 6 mois
Formation complétée	100% des utilisateurs

20. Documentation utilisateur

Document	Emplacement	Description
Guide utilisateur	docs/user-guide.md	Utilisation du dashboard, navigation, alertes
Guide technique	docs/architecture.md	Architecture, composants, flux
Documentation API	Swagger sur chaque backend	Endpoints, paramètres, tests

Document	Emplacement	Description
Documentation IoT	docs/iot-sensor.md	Montage Arduino, bridge, simulation
Documentation Docker	docs/docker-deployment.md	Déploiement, commandes, dépannage
Documentation tests	docs/testing.md	Stratégie, commandes, résultats
Documentation ERP	docs/erp-integration.md	Intégration avec système d'information
Documentation MQTT	docs/mqtt-documentation.md	Topics, commandes, payload
Documentation choix techniques	docs/technical-choices.md	Justification des technologies
Documentation base de données	docs/database.md	Schéma, entités, relations
Documentation changement	docs/change-management.md	Accompagnement au changement
Documentation frontend	docs/frontend.md	Composants, structure, tests
Documentation Jenkins	docs/jenkins-ci.md	Pipeline CI/CD, configuration

21. Matrice de conformité — Grille d'évaluation

Bloc / Compétence	Ce que FutureKawa apporte	Preuve	Niveau visé	Limite
Analyse du besoin	Cahier des charges complet avec besoins fonctionnels, techniques, IoT	Ce rapport, section 3	Niveau 3	Manque entretien client réel
Cahier des charges	Tableau besoins/réponse/statut	Ce rapport, section 3.1	Niveau 3	Quelques statuts "Prévu"
Architecture logicielle	Microservices, backend central + 3 pays, isolation données	Schéma architecture, docs/architecture.md	Niveau 3	Documentation suffisante
Développement backend	4 services NestJS, API REST, Prisma, MQTT	Swagger, code source, tests	Niveau 3	Pas d'authentification

Bloc / Compétence	Ce que FutureKawa apporte	Preuve	Niveau visé	Limite
Développement frontend	React/Vite, dashboard, graphiques, réglage seuils	Frontend fonctionnel, tests (31)	Niveau 3	Peut être enrichi
Base de données	PostgreSQL par pays, Prisma ORM, 3 tables	Schéma Prisma, docs/database.md	Niveau 3	Pas de migrations avancées
IoT / MQTT	Mosquitto par pays, Arduino DHT11, bridge Serial-to-MQTT	Code Arduino, bridge Node.js, simulation	Niveau 3	Bridge USB nécessaire
Tests	104 tests automatisés (Jest + Vitest)	Tests backend/frontend, docs/testing.md	Niveau 3	Couverture à mesurer
CI/CD Jenkins	Jenkinsfile complet, pipeline 9 étapes	Jenkinsfile, docs/jenkins-ci.md	Niveau 2	Preuve build réussi à compléter
Documentation technique	Architecture, choix techniques, API, déploiement, tests, IoT	Dossier docs/complet	Niveau 3	Documentation riche
Documentation utilisateur	Guide utilisateur complet avec dépannage	docs/user-guide.md	Niveau 3	Guide clair et accessible
Solution intégrée type ERP/progiciel	API REST documentées, Swagger, endpoints ERP dédiés (lots, alertes, sync)	docs/erp-integration.md, module erp	Niveau 3	Intégration réelle à développer
Gestion du changement	Plan complet, profils, formation, indicateurs	docs/changement.md	Niveau 3	Plan détaillé et réaliste
Preuves de fonctionnement	Captures à intégrer (Docker, tests, dashboard, Swagger, MQTT, Arduino)	Section 22 de ce rapport	Niveau 2	Certaines captures à compléter
Sécurité	JWT avec rôles ADMIN/OPERATOR, Guards NestJS, architecture isolée par pays, Docker	Module auth, JWT, RolesGuard	Niveau 3	Chiffrement et secrets manager à ajouter
Qualité logicielle	TypeScript, tests, architecture modulaire, documentation	Code structuré, linting, tests	Niveau 2	Pas de coverage minimum défini

Bloc / Compétence	Ce que FutureKawa apporte	Preuve	Niveau visé	Limite
Déploiement Docker	Docker Compose, 12 conteneurs, volumes, healthchecks	docker-compose.yml, docs/docker-deployment.md	Niveau 2	Déploiement fonctionnel
Monitoring / Health	Endpoint GET /api/health avec statut de tous les services	Module health, HealthController	Niveau 3	Monitoring temps réel à ajouter
Notification email	MailHog SMTP, EmailService, alertes déclenchables	docs/mailhog.md, EmailModule	Niveau 3	Envoi automatique à finaliser

22. Preuves à insérer dans le rapport

Preuve	Fichier conseillé	Objectif
Docker containers actifs	screenshots/01-docker-containers.png	Montrer les 11 conteneurs en fonctionnement
Tests 104 passed	screenshots/02-tests-104-passed.png	Prouver que tous les tests passent
Frontend dashboard	screenshots/03-frontend-dashboard.png	Montrer l'interface utilisateur
Swagger central	screenshots/04-swagger-central.png	Documentation API centralisée
Swagger Brazil	screenshots/05-swagger-brazil.png	Documentation API Brésil
Swagger Ecuador	screenshots/06-swagger-ecuador.png	Documentation API Équateur
Swagger Colombia	screenshots/07-swagger-colombia.png	Documentation API Colombie
Jenkins success	screenshots/08-jenkins-success.png	Pipeline CI/CD réussi
Jenkins console output	screenshots/09-jenkins-console-output.png	Logs de build détaillés
MQTT publication	screenshots/10-mqtt-publication.png	Message publié sur Mosquitto
MQTT measurement stored	screenshots/11-mqtt-measurement-stored.png	Mesure stockée en base
MQTT alert created	screenshots/12-mqtt-alert-created.png	Alerte créée automatiquement

Preuve	Fichier conseillé	Objectif
Threshold settings UI	screenshots/13-threshold-settings-ui.png	Interface de réglage des seuils
Arduino serial monitor	screenshots/14-arduino-serial-monitor.png	Données lues par le capteur DHT11
Serial-to-MQTT bridge	screenshots/15-serial-to-mqtt-bridge.png	Bridge en fonctionnement
Auth login JWT	screenshots/16-auth-login.png	Token JWT retourné par POST /api/auth/login

Auth login JWT | screenshots/18-auth-login.png | Token JWT retourné par POST /api/auth/login |
 Auth protected route | screenshots/19-auth-protected.png | Accès refusé sans token |
 Health check | screenshots/20-health-check.png | Rapport de santé du système |
 MailHog inbox | screenshots/21-mailhog-inbox.png | Interface MailHog avec emails reçus (à compléter) |
 MailHog alert email | screenshots/22-mailhog-alert-email.png | Email d'alerte dans MailHog (à compléter) |
 ERP lots endpoint | screenshots/23-erp-lots.png | Lots au format ERP |
 ERP sync | screenshots/24-erp-sync.png | Synchronisation ERP simulée |

Note : Les preuves marquées “à compléter” doivent être ajoutées avant la soutenance. Les captures peuvent être réalisées en lançant l’application et en effectuant les actions correspondantes.

23. Conclusion

Le projet FutureKawa couvre la majorité des exigences du cahier des charges :

- **Architecture cohérente** : microservices avec backend central, isolation par pays, bases PostgreSQL dédiées, brokers MQTT indépendants
- **IoT validé** : prototype fonctionnel avec Arduino UNO + DHT11, bridge Serial-to-MQTT, simulation sans matériel
- **Alertes opérationnelles** : température, humidité, lots périmés, avec seuils configurables par pays
- **Seuils configurables** : modification en temps réel depuis l’interface frontend
- **Authentification JWT** : rôles ADMIN/OPERATOR, endpoints protégés, login API
- **Notification email** : MailHog SMTP intégré, alertes par email démontrables
- **Connecteur ERP** : module ERP simulé avec endpoints dédiés (lots, alertes, sync)
- **Health check** : endpoint de monitoring avec statut des services pays
- **Tests automatisés** : 104 tests backend et frontend, exécutés via Jest et Vitest
- **Documentation complète** : architecture, utilisateur, technique, API Swagger, ERP, IoT, changement, MailHog
- **CI/CD préparé** : Jenkinsfile complet avec pipeline parallélisé

Les limites sont clairement identifiées : - Seuils stockés en mémoire (perdus au redémarrage) - Intégration ERP simulée (pas de connecteur réel) - Preuve Jenkins à compléter

Les évolutions futures sont définies : migration ESP32, déploiement cloud, connecteurs ERP réels, stockage des seuils en base, monitoring Grafana.

24. Annexes

24.1 Commandes Docker

```
# Démarrer tous les services
docker compose up -d --build

# Arrêter tous les services
docker compose down

# Réinitialiser complètement (supprime les volumes)
docker compose down -v

# Voir les conteneurs actifs
docker ps

# Voir les logs d'un service
docker compose logs -f brazil-service

# Exécuter une commande dans un conteneur
docker compose exec brazil-service sh
```

24.2 Commandes MQTT

```
# Publier une mesure normale (Brazil)
mosquitto_pub -h localhost -p 1883 \
  -t "futurekawa/brazil/warehouse-1/measurements" \
  -m '{"warehouseName":"Warehouse Brazil 1","temperature":29,"humidity":55}'

# Publier une mesure hors seuil (Colombia)
mosquitto_pub -h localhost -p 1885 \
  -t "futurekawa/colombia/warehouse-1/measurements" \
  -m '{"warehouseName":"Warehouse Colombia 1","temperature":40,"humidity":20}'

# S'abonner à un topic pour voir les messages
mosquitto_sub -h localhost -p 1883 -t "futurekawa/brazil/warehouse-1/measurements"
```

24.3 Endpoints API principaux

Méthode	URL	Description
GET	/api/central/countries	Liste des pays

Méthode	URL	Description
GET	/api/central/:country	Lots d'un pays
GET	/api/central/:country/effo	Lots triés EFO
GET	/api/central/:country/measurements	Mesures IoT
GET	/api/central/:country/alerts	Alertes
GET	/api/central/:country/alerts/seuils	Seuils d'alerte
PUT	/api/central/:country/alerts/seuils	Modifier les seuils
POST	/api/central/:country/alerts/seuils	Réinitialiser les seuils
POST	/api/auth/login	Authentification JWT
POST	/api/lots	Créer un lot
GET	/api/health	Health check système
GET	/api/central/email-test	Email test MailHog
GET	/api/central/email-test/temperature	Simuler alerte température
GET	/api/central/email-test/humidity	Simuler alerte humidité
GET	/api/erp/lots	Lots format ERP
GET	/api/erp/alerts	Alertes format ERP
POST	/api/erp/sync	Synchronisation ERP

24.4 Ports utilisés

Service	Port
Frontend	5173
Backend Central	3002
Brazil Service	3001
Ecuador Service	3011
Colombia Service	3012
PostgreSQL Brésil	5432
PostgreSQL Équateur	5433
PostgreSQL Colombie	5434
Mosquitto Brésil	1883
Mosquitto Équateur	1884
Mosquitto Colombie	1885
MailHog SMTP	1025
MailHog UI	8025

24.5 Seuils par pays

Pays	Température min	Température max	Humidité min	Humidité max
Brazil	26°C	32°C	53%	57%
Ecuador	28°C	34°C	58%	62%
Colombia	30°C	36°C	30%	45%

24.6 Commandes de test

Tous les tests backend

```
cd backend-country/brazil-service && npm test
cd backend-country/ecuador-service && npm test
cd backend-country/colombia-service && npm test
cd backend-central && npm test
```

Tests frontend

```
cd frontend && npm run test -- --run
```

24.7 Commandes Jenkins

Vérifier la configuration Jenkins

```
java -jar jenkins-cli.jar -s http://localhost:8080/ list-plugins
```

Déclencher un build manuel

```
java -jar jenkins-cli.jar -s http://localhost:8080/ build FutureKawa
```

Voir le statut du dernier build

```
java -jar jenkins-cli.jar -s http://localhost:8080/ console FutureKawa 1
```

24.8 Commandes MailHog (si déployé)

Démarrer MailHog

```
docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

Interface web

http://localhost:8025

Checklist finale — Avant soutenance

- ☐ Captures Docker ajoutées (12 conteneurs actifs)
- ☐ Captures tests ajoutées (104 tests passés)
- ☐ Captures Swagger ajoutées (central + 3 pays)
- ☐ Captures frontend ajoutées (dashboard avec données)
- ☐ Preuve MQTT ajoutée (publication + mesure stockée)
- ☐ Preuve Arduino ajoutée (Serial Monitor + bridge)
- ☐ Preuve Jenkins ajoutée si disponible (build réussi + console)
- ☐ Preuve MailHog ajoutée si disponible (inbox + email)
- ☐ Preuve authentification JWT ajoutée (login + token)
- ☐ Preuve health check ajoutée (endpoint /api/health)

- ☐ Preuve ERP ajoutée (endpoint /api/erp/lots)
- ☐ Rapport exporté en PDF
- ☐ Groupe / équipe complété
- ☐ Auteur(s) complété(s)